

Übungsblatt: Tic-Tac-Toe lokal und verteilt über MQTT

Hintergrund

Im bisherigen Übungsblatt wurde der ESP32 als typischer IoT-Knoten eingeführt: Ein Mikrocontroller erfasst Messwerte, verarbeitet sie lokal und sendet sie über MQTT an andere Systeme.

Aufbauend darauf erweitert dieses Übungsblatt den Fokus von der reinen Sensorübertragung hin zu einer interaktiven, verteilten Anwendung, bei der der ESP32 nicht nur publiziert, sondern auch auf eingehende MQTT-Nachrichten reagiert. Der ESP32 ist für solche Szenarien besonders gut geeignet, weil er WLAN direkt integriert und sich dadurch ohne zusätzliche Netzwerkhardware mit MQTT-Brokern verbinden kann. In dieser Übung wird das bisherige Publish-Modell um Subscribe-Mechanismen ergänzt, sodass ein lokaler Zustand durch Nachrichten anderer Clients beeinflusst werden kann.

Ausgangslage – starten Sie mit <https://wokwi.com/projects/463803024857553921>

Wokwi ist eine browserbasierte Simulationsumgebung für Elektronikprojekte, in der sich unter anderem ESP32-Systeme, WLAN-Kommunikation und typische IoT-Bauteile direkt im Browser entwickeln und testen lassen. In Wokwi können Sie für diese Übung ein 4x4-Keypad und eine WS2812-kompatible LED-Matrix verwenden.

Für die MQTT-Kommunikation wird erneut `public.cloud.shiftr.io` verwendet. Darüber lassen sich Publish-/Subscribe-Szenarien mit mehreren Clients einfach umsetzen und testen.

Zur Beobachtung der Nachrichten sollen zusätzlich **MQTT Explorer** oder **MQTTX** verwendet werden. Da auf dem öffentlichen Broker viele unterschiedliche Topics aktiv sind, besteht eine wichtige Aufgabe darin, das eigene Topic gezielt auszuwählen und die eigenen Nachrichten unter vielen fremden Beiträgen sicher zu identifizieren.

Verwenden Sie als konzeptionellen Startpunkt erneut das einfache MQTT-Sensorbeispiel in Wokwi: MicroPython MQTT Weather Logger (ESP32).

In dieser Übung wird daraus eine interaktive Anwendung: Statt Sensordaten zu übertragen, implementieren Sie ein Tic-Tac-Toe-Spiel. Die Eingabe erfolgt über ein 4x4-Keypad, und der Spielzustand wird über eine LED-Matrix dargestellt.

MQTT-Parameter

Verwenden Sie für die MQTT-Anbindung die folgenden Parameter:

```
# MQTT Server Parameters
MQTT2_CLIENT_ID = "hsrcm-iot-1"
MQTT2_BROKER    = "public.cloud.shiftr.io"
MQTT2_USER      = "public"
MQTT2_PASSWORD  = "public"
# Wählen Sie eine eindeutige Nummer XX
MQTT2_TOPIC     = "hsrcm-iot/XX"
MQTT2_TOPIC_SUB = b"hsrcm-iot/XX"
```

Hinweis: Ersetzen Sie XX durch eine eindeutige Nummer, damit Ihre Gruppe ein eigenes Topic verwendet. Das ist besonders wichtig, weil auf dem öffentlichen Broker sehr viele Topics aktiv sind und Sie Ihre eigenen Spielzüge sonst nur schwer von fremden Nachrichten unterscheiden können.

Vorgegebene Code-Snippets für Subscribe

Verwenden Sie für den Empfang von Nachrichten die folgenden Bausteine:

```
def sub_cb(topic, msg):  
    print("Nachricht empfangen")  
    print("Topic:", topic.decode())  
    print("Payload:", msg.decode())  
  
client2 = MQTTClient(MQTT2_CLIENT_ID, MQTT2_BROKER, user=MQTT2_USER, password=MQTT2_PASSWORD)  
client2.set_callback(sub_cb)  
client2.connect()  
client2.subscribe(MQTT2_TOPIC_SUB)  
  
while True:  
    client2.check_msg()
```

Diese Struktur entspricht dem üblichen MicroPython-Muster für MQTT-Subscribe: Eine Callback-Funktion verarbeitet eingehende Nachrichten, danach wird das Topic abonniert und in der Hauptschleife regelmäßig mit `check_msg()` geprüft, ob neue Nachrichten angekommen sind.

Aufgabe 1: Wokwi-Aufbau erstellen

Erstellen oder erweitern Sie einen neuen Wokwi-Aufbau mit folgenden Komponenten:

- ESP32
- 4x4-Keypad
- WS2812 NeoPixel Compatible LED Matrix

Das Keypad wird mit digitalen Pins des ESP32 verbunden. Die LED-Matrix wird über einen Datenpin angesteuert und als 3x3-Spielfeld genutzt.

Zuordnung der Tasten zum Spielfeld:

- 1 = oben links, 2 = oben Mitte, 3 = oben rechts
- 4 = Mitte links, 5 = Mitte, 6 = Mitte rechts
- 7 = unten links, 8 = unten Mitte, 9 = unten rechts
- * = Spiel zurücksetzen

Erstellen Sie den Aufbau in Wokwi und schreiben Sie einen Code, so dass auf der 3x3 Matrix, die richtigen LEDs aufleuchten auf Knopfdruck. Z.B. Drücken von 1 (oben links) -> LED oben links leuchtet.

Aufgabe 2: Lokales Tic-Tac-Toe implementieren

Implementieren Sie zunächst eine lokale Version des Spiels ohne Netzwerkaustausch, und testen Sie mindestens fünf unterschiedliche Zugfolgen.

Anforderungen

- Beim Drücken einer Zifferntaste von 1 bis 9 soll das entsprechende Feld auf der LED-Matrix gesetzt werden.
- Ist ein Feld bereits belegt, darf dort kein neuer Zug eingetragen werden.
- Die erste Platzierung und danach alle ungeraden Züge werden **grün** dargestellt.
- Die geraden Züge werden **rot** dargestellt.
- Beim Drücken von * wird das Spiel vollständig zurückgesetzt.
- Nach jedem gültigen Zug wird die Anzeige aktualisiert.

Aufgabe 3: Gewinnbedingung prüfen

Erweitern Sie das Spiel um eine Gewinnlogik.

Das Spiel endet, wenn

- drei gleichfarbige Felder in einer vollständigen Zeile liegen,
- drei gleichfarbige Felder in einer vollständigen Spalte liegen,
- drei gleichfarbige Felder auf einer der beiden Diagonalen liegen
- Sobald eine Gewinnbedingung erfüllt ist, darf kein weiterer Input mehr akzeptiert werden. Das Spiel bleibt gesperrt, bis es mit * zurückgesetzt wird.

Aufgabe 4: MQTT-Publish für Spielzüge

Erweitern Sie das Programm nun so, dass jeder gültige lokale Spielzug zusätzlich über MQTT veröffentlicht wird. Verwenden Sie dafür das selbst gewählte Topic hsrn-iot/XX (Wählen Sie für XX eine eindeutige Zahl).

Die Nachricht soll mindestens enthalten:

- das gewählte Spielfeld,
- die laufende Zugnummer,
- optional eine Kennzeichnung des Spielers oder einen Zeitstempel.

Ein mögliches JSON-Format wäre zum Beispiel:

```
{
  "move": 5,
  "turn": 3,
  "player": 1
}
```

Erweitern Sie Ihren Code so, dass jeder gültige lokale Zug publiziert wird. Dokumentieren Sie drei Beispielnachrichten.

Aufgabe 6: MQTT-Subscribe für Remote-Züge - <https://wokwi.com/projects/463793984933828609>

Erweitern Sie das Spiel nun so, dass auch entfernte Spielzüge verarbeitet werden. Andere Spieler, die dasselbe Topic verwenden, sollen dadurch remote mitspielen können.

Anforderungen

- Spieler 1 steuert die ungeraden Platzierungen und damit die **grünen** Felder.
- Spieler 2 steuert die geraden Platzierungen und damit die **roten** Felder.
- Züge werden über MQTT Publish und MQTT Subscribe zwischen den Clients ausgetauscht.
- Beide Seiten sollen nach jedem empfangenen oder lokal erzeugten Zug denselben Spielfeldzustand anzeigen.
- Bereits belegte Felder dürfen weder lokal noch remote überschrieben werden.

Erweitern Sie Ihr Programm um MQTT-Subscribe und die Verarbeitung empfangener Nachrichten unter Verwendung der vorgegebenen Code-Snippets.

Aufgabe 7: Spielzüge mit MQTT Explorer oder MQTTX beobachten - <https://wokwi.com/projects/463911028515334145>

Verbinden Sie sich zusätzlich mit **MQTT Explorer** oder **MQTTX** mit dem Broker public.cloud.shiftr.io . Verwenden Sie dabei dieselben Zugangsdaten und abonnieren Sie gezielt Ihr eigenes Topic hsrn-iot/XX, damit Ihre Spielzüge trotz der vielen anderen aktiven Topics auf dem öffentlichen Broker eindeutig sichtbar bleiben.

- Stellen Sie die Verbindung mit MQTT Explorer oder MQTTX her.
- Beobachten Sie, welche Nachrichten beim lokalen Spielen gesendet werden.
- Prüfen Sie, ob empfangene Remote-Züge ebenfalls korrekt sichtbar sind.
- Dokumentieren Sie, wie Sie Ihr eigenes Topic zwischen vielen anderen Beiträgen identifiziert haben.

Aufgabe 8: Verteiltes Spiel testen

Testen Sie Ihr System mit zwei Clients, die auf dasselbe Topic verbunden sind.

Testszenarien

- Ein lokaler Zug erscheint auf beiden Clients.
- Ein entfernter Zug erscheint auf beiden Clients.
- Bereits belegte Felder werden nicht überschrieben.
- Nach einem Gewinn akzeptieren beide Seiten keine weiteren Züge.
- Nach * wird das Spiel korrekt zurückgesetzt.

Hinweise zur Umsetzung

- Verwenden Sie die MQTT-Grundstruktur aus dem vorherigen Übungsblatt weiter.
- Nutzen Sie für den Spielfeldzustand eine geeignete Datenstruktur, zum Beispiel eine Liste mit neun Einträgen.
- Achten Sie darauf, lokale und empfangene Spielzüge sauber zu unterscheiden.
- Überlegen Sie, wie Sie verhindern, dass ein empfangener Zug sofort erneut publiziert wird und dadurch Nachrichten-Schleifen entstehen.
- Geben Sie über den Serial Monitor zusätzlich Debug-Informationen aus, etwa empfangene Topics, Payloads und abgewiesene ungültige Züge.

Erweiterungen bei verbleibender Zeit

Wenn Sie die Pflichtaufgaben abgeschlossen haben, erweitern Sie Ihr System um mindestens eine zusätzliche Funktion.

Mögliche Erweiterungen sind:

- Ausgabe des Gewinners im Serial Monitor
- separates Reset-Topic
- Spiel-ID in der MQTT-Nachricht
- Schutz vor doppelter Verarbeitung derselben Nachricht
- akustisches Signal bei Gewinn
- Zuschauer-Modus, der nur subscribt